Assignment10 – Natural Language

Given: June 24 Due: June 29

Problem 10.1 (Ambiguity)

1. Explain the concept of ambiguity of natural languages.

Solution: Ambiguity is the phenomenon that in natural languages a single utterance can have multiple meanings.

---

2. Give two examples of different kinds of ambiguity and explain the readings.

Solution: Here are some examples

- "bank" can be a financial institution or a geographical feature.
- In "I saw her duck" the word "duck" can be a verb or a noun.
- "Time flies like an arrow" could be about the preferences of special insects ("time flies") or about the fact that time passes quickly – e.g. in an exam.
- In "Peter saw the man with binoculars", it could be Peter who is using binoculars, or it could be that Peter saw "the man" who had "binoculars".

Problem 10.2 (Language Identification)

You are given an English, a German, a Spanish, and a French text corpus of considerable size, and you want to build a language identification algorithm A for the EU administration. Concretely A takes a string as input and classifies it into one of the four languages $\ell^* \in \{English, German, Spanish, French\}$. The prior probability distribution for the strings being English/German/Spanish/French, is $\langle 0.4, 0.2, 0.15, 0.15\rangle$.

1. How would you proceed to build algorithm A? Specify the general steps and give/derive the formula for computing ℓ given a string $\mathbf{c}_{1:N}$.

Solution:

1. Build a trigram trigram language model $\mathbf{P}(\mathsf{c}_i|\mathbf{c}_{i-2:i-1}, \ell)$ for each candidate language ℓ by counting trigrams in an ℓ-corpus.

2. Apply Bayes' rule and the Markov property to get the most likely language:

$$\begin{aligned}\ell^* &= \operatorname*{argmax}_{\ell}(P(\ell \mid \mathbf{c}_{1:N})) \\ &= \operatorname*{argmax}_{\ell}(P(\ell) \cdot P(\mathbf{c}_{1:N} \mid \ell)) \\ &= \operatorname*{argmax}_{\ell}(P(\ell) \cdot (\prod_{i=1}^{N} P(c_i \mid \mathbf{c}_{i-2:i-1}, \ell)))\end{aligned}$$

Problem 10.3 (Language Models)

1. How can we obtain a trigram model for a natural language?
   Explain the probability distribution involved.

   Solution: We need a corpus of words over L. Then we count how often each trigram occurs in it and use that to estimate the probability distribution $P(T = t)$ of trigrams t.

   ---

2. Explain informally how we can use trigram models to identify the language of a document D.

   Solution: We build a trigram model for each candidate language. Then we use each model to compute the probability of D occurring in that language. We choose the language with the highest probability.

   ---

3. Explain briefly what named entity recognition is.

   Solution: The task of finding, in a text, names of things and deciding what class they belong to.

Problem 10.4 (Part-of-Speech Tagging)

Consider the following corpus

A dog runs. The man sees the dog. The man shouts.

and the following POS tags: D (determiner) N (noun), V (verb), E (end of sentence).

1. Annotate the corpus with POS tags.

Solution: The tags are DNVE DNVDNE DNVE

---

2. Compute the transition and the sensor model from the corpus.
Note: To do so, you have to make reasonable choices for the border states (first and last tag of the text).

Solution: The states are the 4 tags. To use matrices, we order them as D, N, V, E. To handle the border cases, we assume that the text loops: the last tag of the last sentence is succeeded by the first tag of the first sentence. (There are many other choices, we could make. This one has the advantage that we can use the successors of E as the initial state.)
Then the matrix of the transition model is

$$T_{st} = \begin{pmatrix} 0/4 & 4/4 & 0/4 & 0/4 \\ 0/4 & 0/4 & 3/4 & 1/4 \\ 1/3 & 0/3 & 0/3 & 2/3 \\ 3/3 & 0/3 & 0/3 & 0/3 \end{pmatrix}$$

For example, the corpus gives us $T_{ED} = 3/3$ because the tag E occurs 3 times and is followed by D 3 times.
The possible observations are the 7 words and the period, which we order as a, the, dog, man, runs, sees, shouts, ".". Then the matrix of the sensor model is given by

$$S_{tw} = \begin{pmatrix} 1/4 & 3/4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2/4 & 2/4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Note that no column has more than 1 non-zero entry. That is because our small vocabulary has no words that can occur as multiple different parts of speech.

---

3. Now consider the sentence

    The man runs.

Explain informally how you can use the HMM from above to assign POS tags to that sentence.

Solution: We write t_1, t_2, t_3, t_4 for the four tags. We assume $t_0 = E$ for a virtual preceding state.

Thus, the probability distribution for the tag t_1 of the first word is obtained by being a successor of E. (There are other options to obtain the probability distribution for the initial state, such as taking the occurrences of tags over the entire corpus.)
That yields the distribution $T_{Ex} = \langle 1, 0, 0, 0 \rangle$ for t_1, which forces $t_1 = D$. The distribution $T_{Dx} = \langle 0, 1, 0, 0 \rangle$ for t_2 forces $t_2 = N$. Now we have $T_{Nx} = \langle 0, 0, 3/4, 1/4 \rangle$ for t_3, which leaves the options $t_3 = V$ and $t_3 = E$. We continue for each option:

- $t_3 = V$ yields $T_{Vx} = \langle 1/3, 0, 0, 2/3 \rangle$ for t_4. We obtain the total probabilities of $1 \cdot 1 \cdot 3/4 \cdot 2/3 = 1/2$ for the tag sequence DNVE and $1 \cdot 1 \cdot 3/4 \cdot 1/3 = 1/4$ for the tag sequence DNVD. Incorporating the evidence, we get the following:
  - All relevant entries in the sensor model, i.e., $S_{D,\text{The}}$, $S_{N,\text{man}}$, $S_{V,\text{runs}}$ and $S_{E,.}$ are non-zero. That yields an overall non-zero probability for DNVE.
  - The sensor model contains $S_{D,.} = 0$, so the overall probability for DNVD is 0.
- $t_3 = E$ yields $T_{Ex} = \langle 1, 0, 0, 0 \rangle$, which forces $t_4 = D$. We obtain the total probability as $1 \cdot 1 \cdot 1/4 \cdot 1 = 1/4$ for the tag sequence DNED. But the sensor model contains $S_{E,\text{runs}} = 0$. So this branch leads to an overall probability 0 in any case.

So the most probable tag sequence is DNVE.
This calculation only calculates the most probable tag sequence without systematically taking the evidence into account. That suffices in our case due to the extreme simplicity of our vocabulary: every word can only be generated by a single POS, i.e., most entries in the sensor model are 0 anyway. Similarly, most entries in the transition model are 0—that's why all but one overall probabilities calculated above end up being 0.
The Viterbi algorithm is more complicated because it calculates all conditional probabilities systematically. That is relevant when the same word may correspond to multiple POSs so that multiple POS sequences have non-zero probabilities.
Also note that we could also leverage the boundary conditions more, e.g., by forcing $t_4 = E$ (i.e, every text ends with a complete sentence), which already excludes certain tag sequences independent of probabilities.
